GitHub Copilot Deep Dive + Agentic DevOps

GitHub Copilot Customization

Customizing Copilot for your codebase, team, and DevOps workflows

Workshop Objectives

A deep dive into GitHub Copilot - go beyond basics to master advanced features, integrations, and prompt-engineering techniques

- Hands-on labs to incorporate Copilot’s latest capabilities into your daily developer workflow

Explore Agentic DevOps - using AI-driven agents to automate code review, enhancement, security, and more across the SDLC

- Walk away with practical skills and working configurations you can apply to your own projects

Why Customize & Where to Start

- AI writes great code but knows nothing about your project conventions, architecture, or preferred libraries
- Customization closes that gap, so output matches your codebase from the first prompt
- **Seven customization layers, each solving a different problem:**

Custom Instructions
coding standards applied automatically to every request

Prompt Files
reusable slash commands for repeatable tasks

Agent Skills
multi-step workflows with scripts, examples, and templates (open standard)

Custom Agents
specialized AI personas with restricted tools and handoffs

MCP
open protocol connecting AI to external APIs, databases, and cloud services

Hooks
deterministic shell commands at agent lifecycle points (format, block, audit)

Agent Plugins
distributable bundles of all the above

Start with instructions, layer on what you need as your workflows grow

How They Differ

Customization	What It Does	Format	Scope
Custom Instructions	Enforce coding standards	`.instructions.md / copilot-instructions.md`	Always-on or file-pattern based
Prompt Files	One-step slash commands	`.prompt.md`	On-demand
Agent Skills	Multi-step capabilities with scripts	`SKILL.md + resources`	On-demand, portable (VS Code, CLI, coding agent)
Custom Agents	AI personas with tool restrictions	`.agent.md`	Switched manually or via handoffs
MCP	External tools and data sources	`MCP server config`	When task matches a tool description
Hooks	Guaranteed shell command execution	`Hook JSON + scripts`	Lifecycle events (PreToolUse, PostToolUse, etc.)
Agent Plugins	Bundled distribution packages	`plugin.json`	Installed from marketplaces

Instructions guide behavior; hooks guarantee outcomes

Skills are portable workflows; custom agents are specialized personas with controlled tool access

MCP is the bridge to anything outside the editor and terminal

What You Will Build in the Labs

Lab	Topic	Key Artifacts
00	**Custom Instructions**	copilot-instructions.md, file-based instructions for React, Express, testing, CSS
01	**Custom Agents**	Planner → Implementer → Reviewer chain with handoffs; Feature Builder coordinator
02	**Hooks**	PostToolUse auto-format hook, PreToolUse security guard hook
03	**Skills**	Data Seeder skill with utility script, Test Fixture Generator skill
04	**Plugins**	bookfaves-qa-plugin bundling a skill, agent, and hook
05	**MCP Builder**	TypeScript MCP server with 4 tools for a book database
06	**MCP Data**	Feature generation and test fixtures from live MCP data

Same App

All labs use the same Book Favorites app (Express.js + React/Redux + Cypress + Jest)

Progressive

Each lab builds on artifacts from the previous one

Foundation

Labs 00–06 prepare you for advanced DevOps labs (test generation, CI/CD, IaC, SRE)

Custom Instructions

Markdown rules that guide every Copilot response

Always-on and file-based conventions, debugging, and how instructions compose with agents

What Custom Instructions Are

- Custom instructions are Markdown files that define coding standards, conventions, and project context
- Copilot includes them automatically in chat requests, so you write a rule once and it applies everywhere
- Two types: **always-on** (every request) and **file-based** (pattern-matched)
- Also supports `AGENTS.md`, `CLAUDE.md`, and organization-level instructions for cross-tool and cross-repo consistency
- Not used for inline suggestions (completions as you type), only chat and agent mode

How They Work

- **Always-on:** `.github/copilot-instructions.md`, `AGENTS.md`, or `CLAUDE.md` at workspace root
- **File-based:** `.instructions.md` files in `.github/instructions/` with YAML frontmatter
  - `applyTo` glob pattern controls when instructions activate (e.g., `'**/*.jsx'`, `'**/*.test.js'`)
  - Matched against files the agent is currently working on
- **Priority order:** Personal (user-level) > Repository > Organization
- **Generate with AI:** `/create-instruction` for targeted files, `/init` for full workspace
- **Verify:** `/instructions` shows all loaded files
- **Tips:** keep instructions short, explain the "why" behind rules, show good/bad code examples, skip rules linters already enforce

Debugging: Is Copilot Using My Instructions?

- **Quick check:** `/instructions` in the chat input lists all loaded instruction files with their source and status
- **References section:** expand it at the bottom of any chat response to see which instruction files were included in that request
- **Diagnostics view:** click the Gear icon → **Show Agent Debug Logs** → review Custom Instructions section for errors (wrong path, bad YAML, unmatched glob)
- **Common issues:**
  - File not in `.github/instructions/` (or a configured location)
  - Missing or incorrect `applyTo` glob pattern
  - YAML frontmatter syntax error (missing `---` delimiters, bad indentation)
  - Setting `chat.instructionsFilesLocations` excludes the folder
- `/troubleshoot`: ask the AI to analyze the current session, e.g., *`/troubleshoot list all paths you tried to load customizations`*
  - Requires `github.copilot.chat.agentDebugLog.enabled` to be on

Deeper Debugging: Logs and Chat Debug View

Custom Instructions Logs (trace-level):

1. Command Palette → `Developer: Show Logs...` → select `Window`
2. Command Palette → `Developer: Set Log Level...` → select `Trace`
3. Send a prompt, then check Output for `[InstructionsContextComputer]` lines
4. Shows: files found, matches applied, files added to context
5. Set log level back to `Info` when done

Copilot extension logs:

1. Command Palette → `Developer: Set Log Level...` → `Trace` for GitHub Copilot + GitHub Copilot Chat
2. Command Palette → `Output: Show Output Channels` → select `GitHub Copilot` or `GitHub Copilot Chat`
3. Diagnose connection issues, extension errors, unexpected behavior

Chat Debug View (raw LLM inspection):

1. Chat view overflow menu (`...`) → **Show Chat Debug View**
2. Shows full system prompt, user prompt, context, response, and tool invocations for each interaction
3. Expandable sections: System prompt (verify instructions appear), User prompt (#-mentions resolved), Context (files attached), Response (raw model output), Tool responses (MCP/tool inputs and outputs)

Agent Debug Log panel (Preview):

1. Gear icon in Chat view → **Show Agent Debug Logs**
2. Chronological event log: tool calls, LLM requests, token usage, prompt file discovery, errors
3. Can attach a snapshot to a chat conversation to ask the AI to help troubleshoot

How Instructions Feed into Other Customizations

- Instructions define **what coding rules apply**; agents define **a persona** (e.g., Planner, Reviewer) **and restrict which tools that persona can use**
- All instruction files are automatically available to custom agents, skills, and the coding agent
- File-based instructions activate based on what the agent is editing, regardless of which agent is active
- Instruction files can be referenced in prompt files and custom agents via Markdown links
- Instructions + agents + hooks + skills + plugins = **composable customization stack**

Lab 00: Custom Instructions for the Book Favorites App

- **Duration:** ~45 minutes across 3 parts
- **Part 1** (10 min): Enhance the existing `copilot-instructions.md` with project architecture and naming conventions; verify with `/instructions` and the References section
- **Part 2** (20 min): Use `/create-instruction` + Context7 MCP to generate 4 file-based instruction files:
  - `react.instructions.md` → `applyTo: '**/*.jsx'` (functional components, Redux hooks, CSS Modules)
  - `express.instructions.md` → `applyTo: '**/*.js'` (factory function pattern, auth middleware, error format)
  - `testing.instructions.md` → `applyTo: '**/*.{test,spec,cy}.{js,jsx}'` (Jest + Cypress conventions)
  - `css.instructions.md` → `applyTo: '**/*.module.css'` (kebab-case, no !important, CSS custom properties)
- **Part 3** (5 min): Verify all files with `/instructions`, troubleshoot with Diagnostics, understand priority order
- **Bonus:** Security instruction file (OWASP-aligned) and documentation standards instruction file
- All artifacts carry forward into Lab 01 (Custom Agents)

Custom Agents

Specialized AI personas with restricted tools and handoffs

Agent format, tool restrictions, handoff chains, subagent orchestration, and security

What Custom Agents Are

- Custom agents are `.agent.md` files that give the AI a specific persona with its own behavior, tools, and instructions
- Each agent is a preconfigured combination: specialized instructions + restricted tool list + optional model preference
- Without agents, you manually select tools and craft prompts every time; with agents, you switch personas from a dropdown
- Stored in `.github/agents/` (workspace, shared via version control) or `~/.copilot/agents/` (user, personal)
- Also supports `.claude/agents/` for compatibility with Claude Code
- Custom agents work in VS Code, background agents (CLI), and cloud agents

The .agent.md Format

- **YAML frontmatter** defines the configuration:
  - `name` / `description` – identity and placeholder text in the chat input
  - `tools` – restricted list of available tools (e.g., `['search', 'edit/editFiles']`). If a tool is unavailable, it is ignored
  - `model` – single model or prioritized list with fallback (e.g., `['Claude Sonnet 4', 'GPT-4o']`)
  - `agents` – list of subagents this agent can invoke (requires `agent` tool)
  - `user-invocable` – `false` hides it from the dropdown (worker-only subagent)
  - `disable-model-invocation` – `true` prevents other agents from auto-invoking it
  - `handoffs` – guided transitions to other agents with label, target, prompt, and optional auto-send
  - `hooks` (Preview) – lifecycle hooks scoped to this specific agent
- **Markdown body** contains the instructions, rules, output format, and workflow
  - Can reference files via Markdown links and tools via `#tool:<tool-name>`
  - Gets prepended to every user prompt when the agent is active
- **Generate with AI:** `/create-agent` bootstraps from a natural language description; can also extract from a conversation

Handoffs and Subagent Orchestration

- **Handoffs** create guided sequential workflows between agents:
  - Buttons appear after each response; user clicks to switch agent with context carried over
  - Each handoff specifies: `label` (button text), `agent` (target), `prompt` (pre-filled), `send` (auto-submit), `model` (override)
  - Example chain: Planner → Implementer → Reviewer → back to Implementer for fixes
  - User stays in control: reviews output at each step before advancing

- **Subagents** enable automatic delegation within a single conversation:
  - List child agents in the `agents` property; include `agent` in `tools`
  - Coordinator spawns workers mid-conversation; workers report back; coordinator synthesizes
  - Workers set `user-invocable: false` to stay hidden from the dropdown
  - Two orchestration patterns:
    - **Coordinator and Worker**: sequential delegation (plan → architect → implement → QA)
    - **Multi-perspective review**: parallel subagents for independent analysis (correctness, security, quality, architecture)

- **Key difference:** handoffs = manual transitions with user approval; subagents = automatic delegation by the coordinator

Security and Best Practices

- **Principle of least privilege:** give each agent only the tools it needs
  - Read-only agents: `['search', 'search/codebase', 'search/usages', 'web/fetch']`
  - Write agents: add `'edit/editFiles'`, `'edit/createFiles'`, `'read/terminalLastCommand'`
  - Never grant all tools (`'*'`) to security-sensitive agents
- **Review tool lists and instructions** before sharing agents via version control
- Share workspace agents in `.github/agents/` for team consistency; use `chat.agentFilesLocations` for additional folders
- Organization-level agents: define once at the GitHub org level, auto-discovered by all members
- Custom agents inherit all custom instruction files automatically (`.instructions.md` with `applyTo` patterns still activate per file type)
- **Agent Diagnostics:** Select Agent Debug Logs → Diagnostics to verify loaded agents, tool configs, and errors
- Previously called "chat modes"; rename `.chatmode.md` files to `.agent.md`

Lab 01: Custom Agents for the Book Favorites App

- **Duration:** ~65 minutes across 5 parts
- **Part 1** (10 min): Create a **Planner** agent with read-only tools (`search`, `codebase`, `usages`, `web/fetch`); verify tool restrictions prevent file edits
- **Part 2** (15 min): Create **Implementer** (read+write) and **Reviewer** (read-only) agents; connect all three with handoffs (Planner → Implementer → Reviewer → fix loop)
- **Part 3** (10 min): Use `/create-agent` to generate a **Database Migrator** agent from a natural language description; refine tools, add backup and confirmation rules
- **Part 4A** (15 min): Build a **Feature Builder** coordinator with 4 worker subagents (FB Planner, FB Architect, FB Implementer, FB QA); workers use `user-invocable: false`; coordinator auto-delegates sequentially
- **Part 4B** (10 min): Build an **FB Reviewer** that spawns 4 parallel subagents (correctness, quality, security, architecture) for multi-perspective code review
- **Part 5** (5 min): Review agent visibility (`user-invocable` vs `disable-model-invocation`), diagnostics, and how agents feed into hooks (next lab)
- **Bonus:** API Designer agent with OpenAPI generation, multi-model comparison (Quick Answer vs Detailed Answer), TDD pipeline rewiring via handoff changes only

Agent Hooks

Deterministic shell commands at agent lifecycle points

Security enforcement, code formatting, audit trails, and approval controls

What Hooks Are and Why They Matter

- Hooks execute custom shell commands at specific lifecycle points during agent sessions
- Unlike instructions (which guide behavior), hooks **guarantee outcomes** by running your code
- Five key use cases:
  - **Enforce security:** block dangerous commands (`rm -rf`, `DROP TABLE`) before they execute
  - **Automate quality:** run formatters, linters, or tests after file edits
  - **Create audit trails:** log every tool invocation for compliance and debugging
  - **Inject context:** add project info, environment details, or API keys at session start
  - **Control approvals:** auto-approve safe operations, require confirmation for sensitive ones
- Hooks run with the same permissions as VS Code, so review scripts carefully
- Currently in Preview; compatible with Claude Code and Copilot CLI hook formats

Lifecycle Events and How They Fire

- Eight lifecycle events, each firing at a specific point:

Event	When It Fires	Common Use Cases
`SessionStart`	First prompt of a new session	Initialize resources, inject project context
`UserPromptSubmit`	User submits a prompt	Audit requests, inject system context
`PreToolUse`	Before agent invokes any tool	Block dangerous ops, require approval, modify input
`PostToolUse`	After tool completes successfully	Run formatters, log results, trigger follow-ups
`PreCompact`	Before context is compacted	Export context, save state before truncation
`SubagentStart`	Subagent is spawned	Track nested agent usage, init resources
`SubagentStop`	Subagent completes	Aggregate results, cleanup
`Stop`	Agent session ends	Generate reports, cleanup, send notifications

- **PreToolUse** is the primary security enforcement point: it can return `permissionDecision` of `"deny"`, `"ask"`, or `"allow"`
- **PostToolUse** is the primary quality enforcement point: runs after successful tool execution
- **Stop** hooks can block the agent from stopping (e.g., "run tests before finishing"), but check `stop_hook_active` to prevent infinite loops
- Input arrives via **stdin as JSON**; output goes to **stdout as JSON**

Global vs Agent-Scoped Hooks

- **Global hooks** (workspace-wide):
  - JSON files in `.github/hooks/` (e.g., `format.json`, `security.json`)
  - Fire for **every** agent in the workspace
  - Use for organization-wide policies: security enforcement, code formatting, audit logging
  - Also supports `.claude/settings.json` and `~/.copilot/hooks/` (user-level)
  - Configure locations via `chat.hookFilesLocations` setting

- **Agent-scoped hooks** (Preview):
  - Defined in the `hooks` property of an agent's `.agent.md` YAML frontmatter
  - Fire **only** when that specific agent is active (user-selected or invoked as subagent)
  - Run **in addition to** any global hooks for the same event
  - Requires `chat.useCustomAgentHooks: true`
  - Use for agent-specific guardrails: protect data files from the Implementer, enforce review rules on the Reviewer

- **Disabling hooks:** rename `.json` to `.json.disabled` (global) or remove `hooks` from frontmatter (agent-scoped)
- **Generate with AI:** `/create-hook` describes the automation, generates config and script
- **Configure via UI:** `/hooks` or gear icon → Hooks in the Chat view

Hook Configuration and Security

Configuration format (JSON):

```
{
  "hooks": {
    "PreToolUse": [
      { "type": "command", "command": "node
./scripts/guard.js", "timeout": 15 }
    ]
  }
}
```

- **Command properties:** `type` (must be `"command"`), `command`, `timeout` (seconds, default 30), `cwd`, `env`, OS-specific overrides (`windows`, `linux`, `osx`)
- **Exit codes:** 0 = success (parse stdout JSON), 2 = blocking error (stop + show error), other = non-blocking warning
- **PreToolUse output:** `permissionDecision` (`"deny"` / `"ask"` / `"allow"`), `permissionDecisionReason`, `updatedInput`, `additionalContext`
- **PostToolUse output:** `decision: "block"` to halt, `additionalContext` to inject feedback

Security considerations:

- Hooks run with VS Code's full permissions; review all scripts before enabling
- Use `chat.tools.edits.autoApprove` to prevent the agent from editing hook scripts without approval
- Validate and sanitize all stdin input to prevent injection
- Never hardcode secrets in hook scripts; use environment variables
- Workspace hooks take precedence over user hooks for the same event

- **Cross-tool compatibility:** reads Claude Code `.claude/settings.json` and Copilot CLI configs; maps tool names and casing automatically

Lab 02: Hooks for the Book Favorites App

- **Duration:** ~30 minutes across 2 parts
- **Prerequisite:** Custom Agents lab completed (Planner, Implementer, Reviewer, Feature Builder agents in place)
- **Part 1** (15 min): Create two **global hooks** in `.github/hooks/`:
  - `PostToolUse` auto-format hook: runs Prettier on every file the agent edits; uses a Node.js script to parse stdin JSON and extract `filePath`; verify via **GitHub Copilot Chat Hooks** output channel
  - `PreToolUse` security guard hook: blocks dangerous terminal commands (`rm -rf /`, `DROP TABLE`, `npm publish`, `git push --force`); returns `permissionDecision: "deny"` with a clear reason
- **Part 2** (15 min): Create an **agent-scoped hook** using `/create-hook`:
  - PreToolUse hook scoped to the **FB Implementer** agent via `hooks` in its `.agent.md` frontmatter
  - Blocks file edits to `backend/data/` directory; directs users to the Database Migrator agent instead
  - Verify: same edit blocked on FB Implementer, allowed on default Copilot agent
  - Compare global vs agent-scoped: scope, location, use case, and setting requirements
- **Debugging hooks:** Output panel → **GitHub Copilot Chat Hooks** channel shows execution logs, timing, and output for every hook invocation
- Artifacts carry forward into Lab 03 (Agent Skills)

Plan → Generate → Implement

Lab 10

PR-Based Agentic Workflows with Model Tiering

How the Workflow Works

/plan

Large Model

Produces plan.md with architecture decisions, scope, and implementation order. No code written.

→

/generate

Large Model

Reads plan.md and produces implementation.md with atomic, commit-sized tasks in dependency order.

→

TaskRunner

Smaller Model

Executes one task at a time from implementation.md. Each task = one commit. All tasks = one PR.

Large model plans twice (quality) · Small model executes N times (speed & cost)

Lab 10 — What You'll Build

1	**Create /plan prompt file**	Reusable slash command → plan.md
2	**Create /generate prompt file**	Converts plan → implementation.md
3	**Create TaskRunner agent**	Focused agent for task execution
4	**Run the full pipeline**	Plan → Generate → Implement a feature

Outcome: Implement a real feature with atomic commits, clean git history, and model-tiered efficiency

Agent Skills

Reusable capability folders with progressive loading

Specialized workflows, utility scripts, companion resources, and skill chaining

What Agent Skills Are

- Agent Skills are **folders** of instructions, scripts, examples, and resources that Copilot loads on demand
- Unlike instructions (coding guidelines) or agents (personas with tools), skills teach **specialized capabilities and workflows**
- Follow an **open standard** (Agent Skills spec): portable across VS Code, Copilot CLI, and the Copilot coding agent
- Each skill lives in its own directory with a `SKILL.md` file and optional companion resources
- Key benefits:
  - **Specialize:** tailor Copilot for domain-specific tasks (testing, debugging, deployment, data seeding)
  - **Reduce repetition:** create once, use automatically across all conversations
  - **Compose:** combine multiple skills to build complex workflows
  - **Efficient loading:** only relevant content loads into context when needed

How Skills Work: Progressive Loading and the SKILL.md Format

- **Three-level progressive loading** keeps context efficient:
  - **Discovery:** Copilot reads only `name` + `description` from YAML frontmatter
  - **Instructions loading:** Copilot loads the `SKILL.md` body when the task matches (or user types `/skill-name`)
  - **Resource access:** Copilot reads scripts, templates, or examples only when referenced

- **SKILL.md frontmatter** (required fields):
  - `name` – lowercase with hyphens, must match directory name (max 64 chars)
  - `description` – specific about capabilities and use cases so Copilot knows when to load it (max 1024 chars)
  - `argument-hint` – (optional) hint text for the slash command input
  - `user-invocable` – (optional, default true) controls `/` menu visibility
  - `disable-model-invocation` – (optional, default false) prevents automatic loading

- **Body:** Markdown instructions with step-by-step workflows, schemas, and relative-path references to companion files

- **Directory structure:**

```
.github/skills/my-skill/
├── SKILL.md            ← required
├── templates.md        ← companion reference (loaded on demand)
└── scripts/
    └── generate.js     ← utility script (executed, not read into context)
```

Slash Commands, Visibility, and Sharing

- **Slash commands:** every skill is automatically available as `/skill-name` in the chat input
  - Add context after the command: `/seeding-test-data small edge-cases`
  - Also triggered automatically when the task matches the description

- **Visibility control** via frontmatter:

Setting	/ menu	Auto-loaded by model	Use case
Default (both omitted)	Yes	Yes	General-purpose skills
`user-invocable: false`	No	Yes	Background knowledge loaded when relevant
`disable-model-invocation: true`	Yes	No	On-demand only
Both set	No	No	Effectively disabled

- **Skill locations:**
  - **Project:** `.github/skills/`, `.claude/skills/`, `.agents/skills/`
  - **Personal:** `~/.copilot/skills/`, `~/.claude/skills/`
  - Custom: configure via `chat.agentSkillsLocations` setting

- **Sharing:** commit to `.github/skills/` for team use; browse community skills at github/awesome-copilot and anthropics/skills
- **Generate with AI:** `/create-skill` bootstraps from a description; can also extract from an ongoing conversation
- **Extensions:** can contribute skills via `chatSkills` in `package.json`
- **Verify:** `/skills` opens the Configure Skills menu; Diagnostics shows load status and errors

How Skills Complement Instructions, Agents, and Hooks

- **Instructions** define coding rules; **agents** define personas with tools; **hooks** guarantee deterministic actions; **skills** teach reusable capabilities

- Skills work **alongside** all three:
  - Instructions still apply when a skill is active (e.g., `testing.instructions.md` conventions apply to test fixtures a skill generates)
  - Agents can load skills automatically (e.g., a Test Writer agent leverages a test fixture skill)
  - Hooks still fire when a skill triggers actions (e.g., auto-format hook runs on files a skill creates)

- **Skill chaining:** skills can build on each other (Part 1 seeds data, Part 2 generates tests against it)

- **Utility scripts vs generated code:** pre-made scripts are more reliable and token-efficient than having the AI generate equivalent code each time

- Skills are the most **portable** customization: VS Code + CLI + coding agent + any skills-compatible agent

Lab 03: Agent Skills for the Book Favorites App

- **Duration:** ~60 minutes across 2 parts
- **Prerequisite:** Custom Agents lab completed (Planner, Implementer, Reviewer, Feature Builder agents in place)

- **Part 1** (30 min): Generate a **Test Fixture Generator** skill with `/create-skill`:
  - Single detailed prompt generates `SKILL.md`, `templates.md` (Jest/supertest templates), and `scripts/generate-fixtures.js`
  - Review against checklist: name matches directory, description has trigger terms, body under 200 lines, relative-path references
  - Test the skill: inspect `backend/routes/books.js` for real endpoints, generate tests only for what exists, run `npm run test:backend`
  - Generate fixtures for favorites API and edge-case book data (special characters, boundary years, long titles)
  - Experiment with `user-invocable` and `disable-model-invocation` frontmatter controls

- **Part 2** (30 min): Extract and install community skills:
  - Introduce a deliberate bug, debug with Copilot, extract the debugging procedure as a reusable skill via `/create-skill`
  - Install three community skills from **github/awesome-copilot**: `review-and-refactor`, `conventional-commit`, `javascript-typescript-jest`
  - Observe **skill composition**: generate tests → improve with Jest best practices → review against project conventions → commit with conventional format

Skills vs Subagents

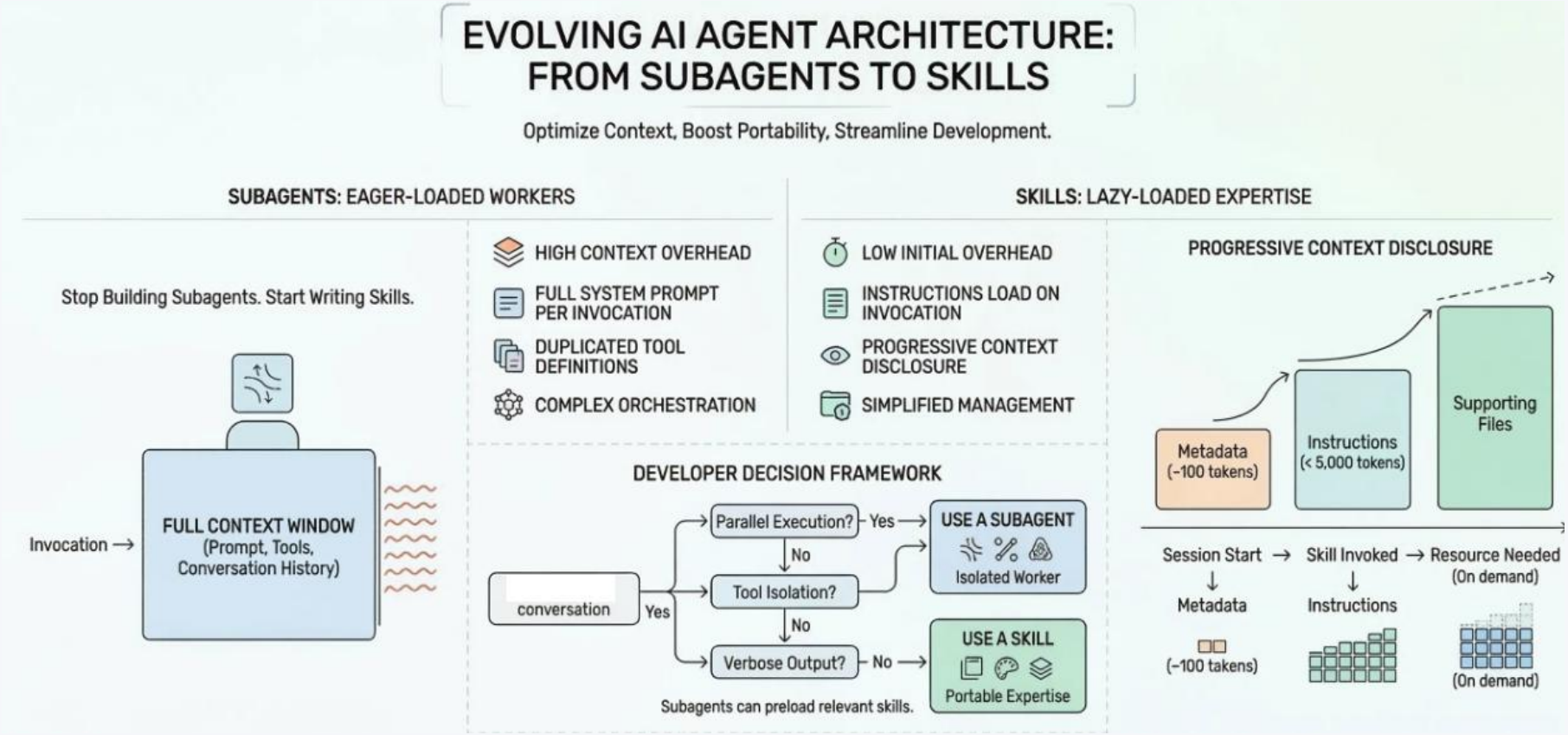
Developer decision framework for context-efficient architecture

When to use portable expertise vs isolated workers, and how they combine

Subagents vs Skills: How They Use Your Context Budget

- **Subagents are eager-loaded**: every invocation spins up a fresh context window with the full system prompt, tool definitions, and conversation context
- Ten subagents = ten separate context windows, each paying the full cost upfront
- **Skills are lazy-loaded** via progressive context disclosure:
  - **Session start:** load metadata only (~100 tokens per skill)
  - **Skill invoked:** load full instructions (< 5,000 tokens)
  - **Resource needed:** load supporting files on demand (scripts, templates, references)
- You can have dozens of skills installed and pay almost nothing until one is actually called
- Skill descriptions consume approximately 2% of the context window
- Compare that to spinning up a subagent with a multi-thousand-token system prompt every time you need a code review

The Decision Framework: Skill or Subagent?



Why Skills Win for Most Use Cases

- **No orchestration logic needed:** folder structure is the routing
  - Skills in `.github/skills/` (or `.claude/skills/`) auto-discovered based on location
  - Monorepo: frontend skills load when editing frontend code, backend skills load when editing backend code
  - Eliminates: routing logic, conditional dispatch, subagent orchestration code

- **Portable across 30+ tools:** Claude Code, VS Code Copilot, Cursor, Gemini CLI, JetBrains Junie, and more
  - Base spec (`name`, `description`, Markdown instructions) works everywhere
  - Tool-specific extensions (hooks, model selection, `disable-model-invocation`) are additive

- **No framework, no SDK:** a skill is a directory with a Markdown file

- **Context budget savings:** lazy loading means skills scale without proportional context cost

- **Simplified architecture:** replaced code review, generation, research, and style consistency subagents with skills, same capabilities, fraction of the overhead

Applying This to Our Workshop

- **Labs 00–02** (Instructions, Agents, Hooks): built the customization foundation

- **Lab 03** (Skills): created Data Seeder and Test Fixture Generator as **skills**, not subagents
  - Progressive disclosure: SKILL.md overview + companion files loaded on demand
  - Utility scripts executed directly, not read into context (token-efficient)
  - Skills chain with each other: seed data → generate fixtures against it

- **Lab 01 Part 4A** (Feature Builder): used **subagents** for the coordinator/worker pattern
  - FB Planner, FB Architect, FB Implementer, FB QA as isolated workers
  - Right choice: needs tool isolation (Implementer has write tools, QA has read-only) and parallel-capable architecture

- **The combination:** Feature Builder subagents can **preload skills**, getting the coordination isolation of subagents with the context efficiency of skills

- **Rule of thumb:** start with a skill; upgrade to a subagent only when you need isolation or parallelism

MCP (Model Context Protocol)

Connecting AI models to external tools and live data

Build, test, and exercise MCP servers for databases, APIs, and cloud services

What MCP Is and Why It Matters

- **Model Context Protocol (MCP)** is an open standard for connecting AI models to external tools and services
- Without MCP, the AI can only work with code in your editor and the terminal
- With MCP, the AI can query databases, call APIs, interact with cloud services, browse the web, and access any external system
- MCP servers provide four capabilities:
  - **Tools** – executable actions the AI can invoke (query, create, update, delete)
  - **Resources** – read-only context attached to prompts (files, database tables, API responses)
  - **Prompts** – preconfigured prompt templates tailored to server capabilities
  - **MCP Apps** – interactive UI components (forms, visualizations) rendered directly in chat
- MCP servers run locally (stdio) or remotely (HTTP), and can be sandboxed for security
- The same MCP server works across VS Code, CLI, coding agent, and other MCP-compatible tools

How to Add and Configure MCP Servers

- **Three ways to add an MCP server:**
  - **MCP Gallery:** Extensions view → search `@mcp` → install from gallery (one click)
  - **mcp.json configuration:** create `.vscode/mcp.json` (workspace, shared via source control) or user profile config
  - **Command line:** `code --add-mcp '{"name":"server","command":"npx","args":[...]}'`

- **mcp.json format:**

```
{
  "servers": {
    "book-database": {
      "type": "stdio",
      "command": "node",
      "args": ["book-database-mcp-server/dist/index.js"]
    }
  }
}
```

- **Server types:** `stdio` (local process) or `http` (remote/streamable HTTP)
- **Security:** never hardcode API keys; use input variables or `.env` files
- **Trust:** VS Code prompts for trust confirmation before starting a new server
- **Manage:** start/stop/restart via Command Palette (`MCP: List Servers`), Extensions view, or mcp.json code lenses
- **Organization control:** centrally manage MCP server access via GitHub enterprise policies

How MCP Fits into the Customization Stack

- MCP is the **only customization** that connects the AI to systems outside the editor and terminal

- Complements the other layers:
  - **Instructions** define coding rules → MCP provides external data to follow when generating code
  - **Agents** define personas with tool restrictions → agents can include MCP tools in their `tools` list
  - **Hooks** run at lifecycle points → hooks can validate MCP tool inputs/outputs
  - **Skills** teach capabilities → skills can reference MCP tools via `#tool:` syntax
  - **Plugins** bundle everything → plugins can include MCP server configurations

- **Use cases in development workflows:**
  - Pull live data from databases or APIs during code generation (no placeholder values)
  - Generate test fixtures from real data sources
  - Query documentation servers for up-to-date API references (Context7)
  - Interact with cloud infrastructure (Azure, AWS, GCP)
  - Automate DevOps workflows (GitHub issues, PRs, deployments)

- Community ecosystem: MCP Gallery, github/awesome-copilot, and third-party MCP servers

Lab 05: Build an MCP Server with the MCP Builder Skill

- **Duration:** 45–60 minutes across 5 exercises

- **Exercise 1** (10 min): Install the **Anthropic MCP Builder skill** via `/create-skill` or manual download; verify 4-phase workflow (Research → Implement → Review → Evaluate)

- **Exercise 2** (10 min): Prompt Copilot to scaffold a TypeScript MCP server in `book-database-mcp-server/`; verify project structure, `package.json` (`type: module`, MCP SDK, Zod), `tsconfig.json` (`strict: true`)

- **Exercise 3** (15 min): Implement 4 tools (`get_book_by_isbn`, `get_book_by_title`, `get_books_by_titles`, `get_books_by_isbn_list`) using `server.registerTool()` (modern API); Zod `.strict()` schemas with `.describe()`; tool annotations (`readOnlyHint`, `destructiveHint`, `idempotentHint`)

- **Exercise 4** (10 min): Test with **MCP Inspector** (`npx @modelcontextprotocol/inspector`); verify all 4 tools, valid inputs, not-found errors, Zod validation errors

- **Exercise 5** (10 min): Wire into VS Code via `.vscode/mcp.json`; start server from Command Palette; test from Copilot Chat with real book queries

- **Key takeaway:** always test MCP tools with Inspector before connecting to an AI client
- This server is used in Lab 06 for feature generation and test fixture creation from live MCP data

Inside the Book Database MCP Server

- **Architecture:** 6 source files in a clean layered structure

```
src/
├── index.ts          ← Entry point: McpServer + StdioServerTransport
├── types.ts          ← TypeScript interfaces (Book, BookDetails, BookWithDetails)
├── constants.ts      ← Shared constants (CHARACTER_LIMIT)
├── schemas/
│   └── book-schemas.ts  ← Zod schemas with .strict() and .describe()
├── services/
│   └── book-service.ts  ← Data access: load JSON, merge, query
└── tools/
    └── book-tools.ts    ← Tool registrations via server.registerTool()
```

- **MCP SDK abstractions** (2 key imports from `@modelcontextprotocol/sdk`):
  - `McpServer` – creates the server instance with name and version
  - `StdioServerTransport` – handles JSON-RPC over stdin/stdout
- **Tool registration** uses `server.registerTool()` (modern API, not deprecated `server.tool()`):
  - Tool name, metadata object (title, description, inputSchema, annotations), and async handler
  - **Annotations:** `readOnlyHint: true`, `destructiveHint: false`, `idempotentHint: true`, `openWorldHint: false`
- **Zod validation:** schemas use `.strict()` (reject extra fields), `.describe()` (field-level docs for AI discoverability), `.min()/.max()` for bounds
- **Service layer:** loads JSON data once at startup into a `Map` for O(1) ISBN lookups; merges `books.json` + `books-details.json` via `mergeBookWithDetails()`
- **Response formatting:** Markdown output with truncation at 25K characters for large result sets

Lab 06: Exercise the MCP Server in Real Workflows

- **Duration:** 45–60 minutes across 3 exercises
- **Prerequisite:** Lab 05 complete (MCP server built, configured in `.vscode/mcp.json`, and running)

- **Exercise 1 – Feature generation from MCP data** (20 min):
  - Prompt Copilot to look up 5 classic novels via MCP tools, then generate a **Staff Picks** endpoint (`GET /staff-picks`)
  - Generated route uses real titles, authors, ISBNs, summaries from the MCP catalog, not hallucinated values
  - Wire the route into `routes/index.js`, restart backend, verify with `curl`

- **Exercise 2 – Test data generation from MCP** (15 min):
  - Look up books by ISBN via MCP tools, generate a JSON fixture file with real catalog data
  - Generate a Jest test file that validates the Staff Picks endpoint against the fixture
  - Cross-check: Copilot verifies fixture data matches MCP catalog exactly

- **Exercise 3 – Extend the server with a new tool** (20 min):
  - Add `get_books_by_author` tool (case-insensitive partial match)
  - Full iteration cycle: implement schema + service + tool → `npm run build` → test with MCP Inspector → restart in VS Code → test from Copilot Chat
  - Multi-tool query: find by author, then look up full details by ISBN, then answer a question about the data

MCP: From Build to Daily Use

- **Lab 05 / MCP Builder** taught you to **build** an MCP server: scaffold, implement tools, test with Inspector, wire into VS Code

- **Lab 06 / MCP User** taught you to **use** it: feature generation, test fixtures, and extending with new tools

- **The pattern repeats for any data source:**
  - Database → MCP server → Copilot queries real data during code generation
  - REST API → MCP server → Copilot calls endpoints for live context
  - Cloud service → MCP server → Copilot interacts with infrastructure

- **Three key takeaways:**
  - Always test MCP tools with Inspector before connecting to an AI client
  - MCP eliminates hallucinated data: the AI pulls from authoritative sources
  - The implement → build → test → integrate cycle is fast once the server pattern is established

- **What's next:** MCP servers compose with all other customizations (instructions, agents, hooks, skills, plugins)

GitHub Copilot Coding Agent

Autonomous background development on GitHub.com

Assign an issue to Copilot, get a draft PR back, iterate via PR comments

What the Coding Agent Is

- Copilot coding agent works **autonomously in the background** to complete development tasks, like a human developer
- Runs on **GitHub.com** (not in your IDE) in an ephemeral GitHub Actions–powered environment
- Workflow: assign an issue to Copilot (or prompt from VS Code Cloud mode) → agent analyzes repo → generates code → opens a draft PR → requests your review
- Capable of fixing bugs, implementing features, improving test coverage, updating docs, addressing tech debt
- You **steer via PR reviews**: leave comments, agent iterates and pushes new commits to the same PR
- Every step is tracked in commits and session logs for full transparency
- Uses GitHub Actions minutes and Copilot premium requests (within monthly allowances, no extra cost)

How It Works: Setup and Customization

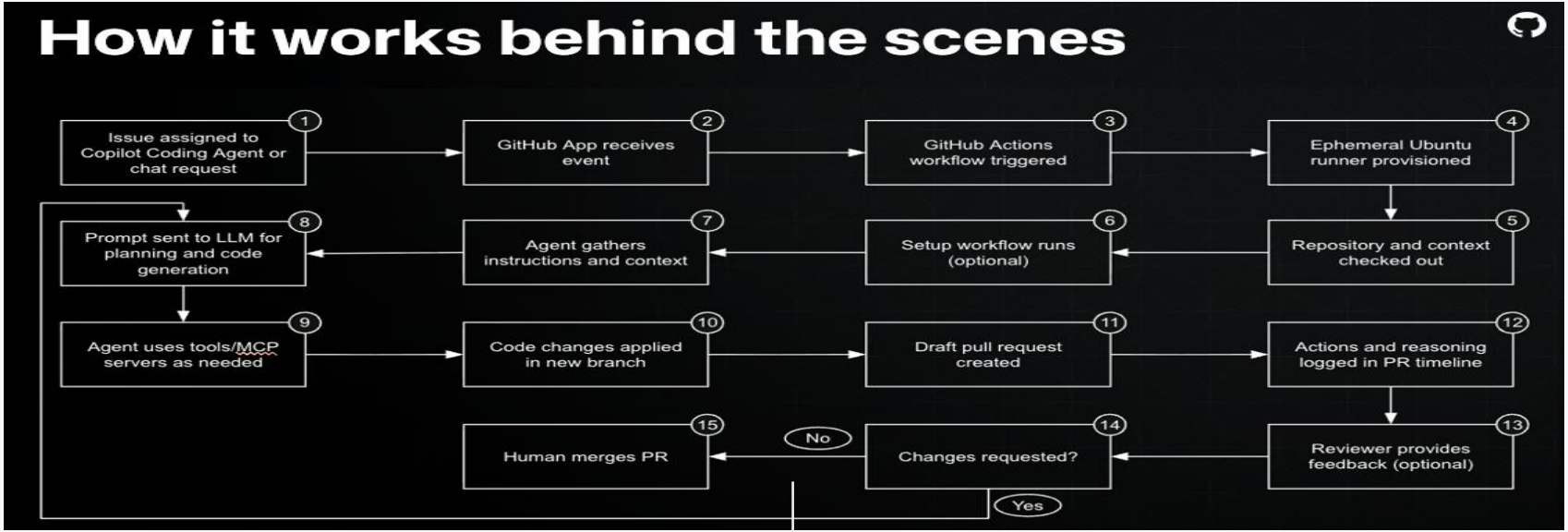
- **Two required files** in the repository:
  - `.github/workflows/copilot-setup-steps.yml` – environment setup (npm install, build, etc.)
  - `.github/copilot-instructions.md` – project conventions the agent follows

- **Three ways to trigger:**
  - Assign a GitHub Issue to `@copilot`
  - Prompt from VS Code Agent Mode → Cloud
  - Mention `@copilot` in a PR comment to request changes on an existing PR

- **Customization layers** (all apply to the coding agent):
  - Custom instructions (`.github/copilot-instructions.md`, `AGENTS.md`)
  - Custom agents (`.agent.md` files for specialized personas)
  - Agent skills (portable `SKILL.md` capabilities)
  - Hooks (lifecycle shell commands)
  - MCP servers (external tools and data sources, configured in repo Settings → Copilot → Coding agent)

- **Security:** sandboxed environment, read-only repo access, pushes only to `copilot/` branches, CodeQL + secret scanning + dependency checks built in

GitHub Coding Agent Workflow



15-step workflow:

1. Issue assigned to Copilot or chat request submitted
2. GitHub App receives webhook event
3. GitHub Actions workflow triggered
4. Ephemeral Ubuntu runner provisioned
5. Repository and context checked out
6. Setup workflow runs (copilot-setup-steps.yml)
7. Agent gathers instructions and context
8. Prompt sent to LLM for planning and code generation
9. Agent uses tools/MCP servers as needed
10. Code changes applied in a new copilot/ branch
11. Draft pull request created
12. Actions and reasoning logged in PR timeline
13. Reviewer provides feedback (optional)
14. Changes requested? If yes, agent iterates (back to 8)
15. Human merges PR

Coding Agent vs Agent Mode

Aspect	Coding Agent	Agent Mode (IDE)
Where it runs	GitHub.com (ephemeral Actions runner)	Local IDE (VS Code)
Interaction	Asynchronous: assign task, come back to review PR	Synchronous: watch changes in real time
Output	Draft PR with commits, timeline, and session logs	Direct file edits in your workspace
Iteration	PR comments trigger new commits	Chat conversation continues
Visibility	Full team can see PR, timeline, and all changes	Only visible to the developer in the IDE
Best for	Backlog items, routine fixes, async delegation	Complex features, interactive exploration, debugging

- **They complement each other:** use agent mode for interactive work, coding agent for background delegation
- Coding agent can start a task that you pick up and continue locally
- Create specialized custom agents for different coding agent tasks (frontend, testing, docs)

Best Practices for Effective Use

- **Write clear, specific issue descriptions:** the agent uses the issue body as its primary prompt
- **Decompose complex features into linked issues:** agent reads linked issues via GitHub MCP and coordinates a full-stack PR
- **Invest in custom instructions:** the more the agent knows about your conventions, the better the output
- **Use copilot-setup-steps.yml wisely:** include build, lint, and test commands so the agent validates its own work
- **Review PRs carefully:** the agent is an “outside collaborator”, so you are responsible for merging
- **Iterate via PR comments:** be specific (“add input validation to the author field”) not vague (“make it better”)
- **Use Copilot as PR reviewer:** assign Copilot as reviewer for automated code quality feedback
- **Start small:** backlog items, test coverage, documentation, refactoring. Build trust before delegating complex features
- **Check session logs:** the Copilot Coding Agent timeline shows every decision, tool call, and error

Lab 07: Coding Agent Exercises

- **Duration:** ~35 minutes across 2 exercises (agent works asynchronously, move between labs while waiting)
- **Exercise 1 – Prompt-to-PR lifecycle + iteration** (20 min):
  - Submit prompt from VS Code Agent Mode (Cloud): “Add a Clear All Favorites button with confirmation dialog”
  - Agent creates PR directly (no issue needed in Cloud mode)
  - Monitor workflow in Actions tab: runner provisioning, checkout, setup steps, agent activity
  - Review PR: body, timeline, files changed (backend route + frontend component + tests)
  - **Iterate via PR comment:** “Add a toast notification confirming favorites cleared”
  - Agent reads comment, pushes new commit to same PR
  - Assign Copilot as PR reviewer for automated code quality feedback

- **Exercise 2 – Multi-issue coordination** (15 min):
  - Create 3 linked issues: frontend UI, backend API, and main “Book Reviews” feature issue
  - Assign main issue to Copilot; verify MCP server configured in repo Settings → Copilot → Coding agent
  - Agent reads linked issues via GitHub MCP, coordinates a single full-stack PR
  - Verify MCP tool calls in the Copilot Coding Agent timeline (`get_issue`, `issue_read`)
  - Move to next lab while agent works (~10 min), return to review PR

Capstone Labs

Greenfield and Brownfield projects showcasing the full AI pipeline

Feature Flag Service sprint (Lab 08) and Book Quotes feature sprint (Lab 09)

Lab 08: Greenfield – Feature Flag Service

- **Scenario:** Build a production-grade feature flag service from scratch in 60 minutes with 8 REST endpoints
- **What you build:** Express.js API (8 endpoints with JWT auth, audit trail, input validation) + React admin dashboard + Jest tests
- **Define conventions first:** Always-on + file-based instructions in .github/ before writing code
- **Build MCP schema server:** Canonical data source with get_flag_schema, get_audit_log_schema, get_sample_flags tools
- **Result:** Production-grade microservice with every file following the same conventions automatically

Lab 08: Greenfield – The Four-Phase Sprint

- **Phase 1:** MCP schema server ready; agents can query real data, not hallucinations
- **Phase 2: Plan** – Planner (read-only) analyzes requirements, queries MCP, produces structured plan with security considerations
- **Phase 3: Implement** – Implementer (write) follows plan, uses skills for scaffolding, hooks auto-format + safety guard, MCP feeds real data
- **Phase 4: Review** – Reviewer (security-focused) audits OWASP, auth, validation, audit trail; fix loop if critical findings exist
- **Orchestration:** All phases enforced by instructions + hooks + skills + handoffs. Every file follows the same standards because the pipeline enforced them.

Lab 09: Brownfield – Book Quotes Feature

- **Scenario:** Ship a Book Quotes feature today on the existing Book Favorites app (not greenfield – conventions already exist)
- **What you build:** 4 API endpoints + React component with blockquotes + Redux slice + Jest tests + security review
- **Key difference:** Conventions already defined in Labs 00-01; your challenge is to follow existing patterns, not create new ones
- **Extend MCP:** Add get_quotes_by_isbn, get_random_quote tools with 15-20 real quotes
- **Add guardrails:** Create quotes-specific instruction file (applyTo: '**/*quote*.*') + agent-scoped data protection hook
- **Result:** New feature fits seamlessly with existing codebase because instructions enforced matching patterns

Lab 09: Brownfield – Reuse, Iterate, Deliver

- **Phase 2: Plan** – Planner calls MCP for quote data shape, produces plan following reusable 8-step skill checklist (same skill as Lab 08)
- **Phase 3: Implement** – Implementer follows the plan; creates route, Redux slice, component, CSS, tests; MCP feeds real quote data into fixtures
- **Phase 4: Review** – Reviewer produces structured security report; fix loop if critical findings exist
- **The difference:** Conventions, agents, hooks, and skills all inherited from earlier work – this sprint focuses on delivery, not setup
- **Same quality baseline:** MCP eliminates hallucinations. Instructions enforce patterns. Hooks guarantee outcomes. Skills structure the work. Handoffs enable iteration.